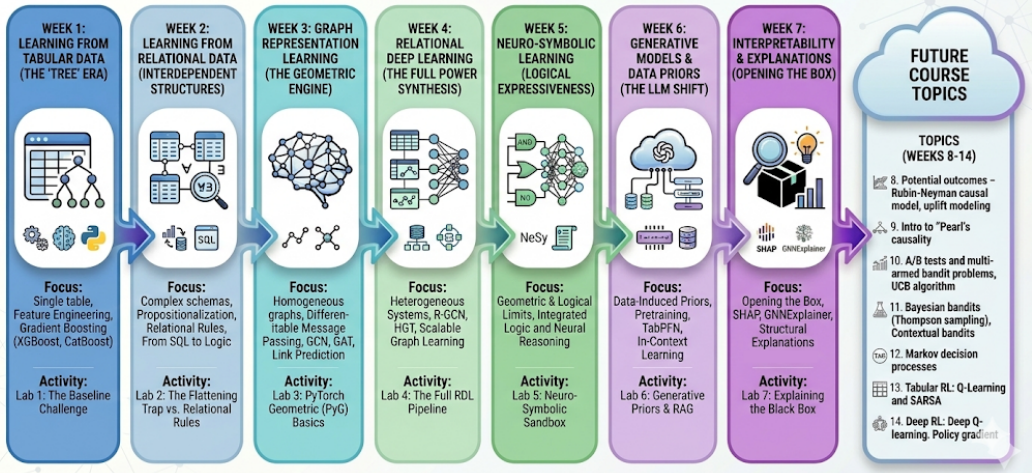


LEARNING FROM STRUCTURED DATA: A 7-WEEK JOURNEY

Narrative Arc: Increasing Structure → Increasing Inductive Bias → Increasing Reasoning Power



Course Map of Week 4: Relational Deep Learning

- 1 The Rosetta Stone – Unifying Logic and Geometry
- 2 The Limits of Geometry – 1-WL Test and Topological Blindness
- 3 The Logic of Graphs – C_2 Logic and the GNN Boundary
- 4 The Precursors – From Rigid Gates to Logic as Loss
- 5 Breaking the Boundary – The Relational Synthesis

Welcome to Week 5: Neuro-Symbolic Learning

Recall Week 2 (Relational Logic):

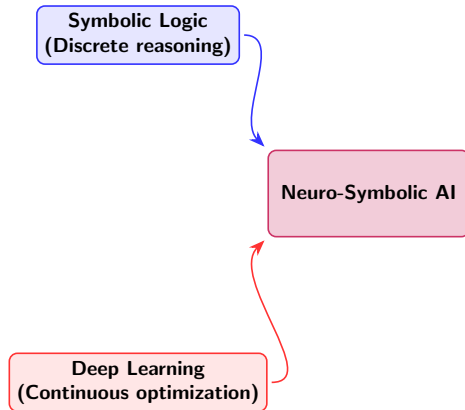
- Discrete, interpretable, rigid (SQL).
- *Flaw*: Fails on continuous noise/gradients.

Recall Week 3 (Geometric Deep Learning):

- Continuous vectors, powerful optimization.
- *Flaw*: Black-box, topologically blind.

Week 5 (Neuro-Symbolic Learning):

- **Goal**: Inject discrete logic into continuous pipelines as a structural regularizer.



Early Motivation: Knowledge Graph Embeddings

The KGE Approach (e.g., TransE)

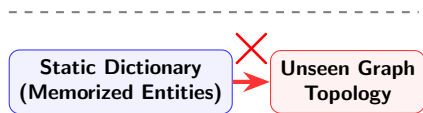
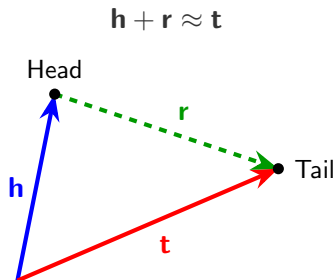
- Long before GNNs, researchers desired a link between logical facts and neural vectors.
- Goal: Learn continuous representations where logic acts via geometry.

The TransE Objective:

- For a factual triplet (*head*, *relation*, *tail*):
- $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$
- Example: $\mathbf{v}_{King} - \mathbf{v}_{Man} + \mathbf{v}_{Woman} \approx \mathbf{v}_{Queen}$

The Limitation: Transductive Lookups

- KGEs proved logic and vectors can coexist.
- *However:* They are shallow lookup dictionaries.



Fails at Inductive Reasoning!

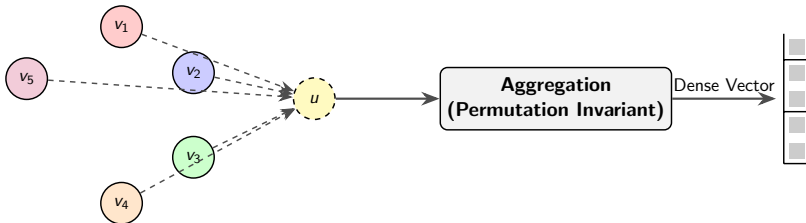
The Shared Challenge: The Multiset Problem

The Fundamental Data Structure:

- Graphs and Relational DBs lack fixed order and dimensionality.
- Nodes possess a *variable-sized multiset* of neighbors.

The Universal Requirement: Permutation-Invariant Aggregation

- To learn, we must reduce a set of N interdependent elements into a single, fixed-size representation.
- How we perform this aggregation defines the computational paradigm.



Perspectives 1 & 2: Databases and Logic

Perspective 1: Relational Databases

- **Mechanism:** SQL GROUP BY + aggregates.
- Collapses 1-to-many relationships into a static table.
- *Concept:* Manual “Eager Propositionalization”.

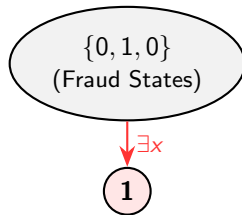
<i>id</i>	<i>val</i>
1	10
1	15
1	5

↓ GROUP BY
SUM()

<i>id</i>	<i>sum(val)</i>
1	30

Perspective 2: Formal Logic

- **Mechanism:** First-Order Quantification ($\exists, \forall$).
- Reduces a set of logical groundings into a Boolean value.
- e.g., $\exists$ trans: Is *any* related element a fraudster?



Commonality: Both apply hard, discrete, human-engineered reductions to multisets.

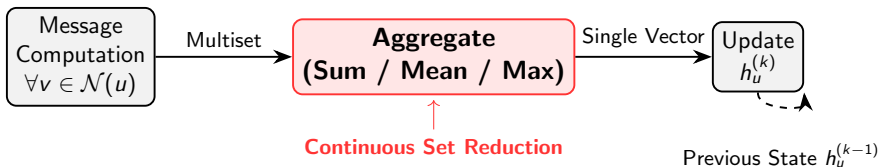
Perspective 3: Geometric Deep Learning

The Message Passing Neural Network (MPNN) Framework:

$$h_u^{(k)} = \text{Update} \left(h_u^{(k-1)}, \text{Aggregate} \left(\{ \text{Message}(h_v^{(k-1)}) : v \in \mathcal{N}(u) \} \right) \right)$$

The Continuous Analog:

- The Aggregate function (e.g., Sum, Mean, Max pooling) operates over the neighborhood multiset $\mathcal{N}(u)$.
- It is a differentiable generalization of both the GROUP BY and logical $\forall/\exists$.



The Rosetta Stone: Unifying the Perspectives

The Rosetta Stone of Structured Data

Discipline	The Multiset Operator	Output Domain
Relational DBs	SQL GROUP BY	Static Scalars
Formal Logic	Quantifiers ($\exists, \forall$)	Boolean $\{0, 1\}$
Geometric DL	GNN Aggregate()	Continuous $\mathbb{R}^d$

The Unifying Insight

- GNNs execute a form of “**soft, differentiable propositionalization.**”
- Instead of manually writing a query, the GNN *learns* the optimal aggregation.

Engagement Question *If GNN aggregation is mathematically equivalent to a continuous SQL SUM over a neighborhood, what happens when two completely different graph topologies happen to sum to the exact same vector?*

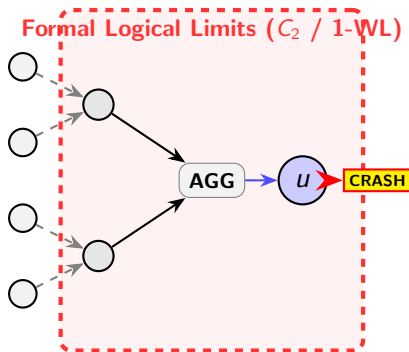
The Implication: Shifting to Formal Mathematics

The Expressiveness Limit

- If GNN aggregation is mathematically analogous to logical quantification...
- ...then the **absolute expressive ceiling** of a GNN is strictly bounded by the formal limitations of the logical language it mimics.

The Pedagogical Shift

- We can no longer treat Neural Networks solely as an empirical discipline.
- We must study them as a **formal mathematical discipline**.
- *Next:* We measure exactly what GNNs are mathematically blind to.



The Graph Isomorphism Problem

The Ultimate Expressivity Test

- Can a GNN distinguish any two graphs?
Maps to the Graph Isomorphism Problem.

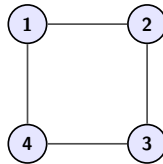
Isomorphic Graphs:

- Isomorphic graphs share identical topology, despite arbitrary node ordering/layout.

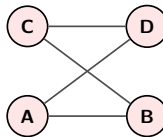
The GNN Goal:

- Map isomorphic graphs to the same embedding; non-isomorphic to different ones.

G_1 (4-Cycle)



✓
ISOMORPHIC



G_2 (4-Cycle)

The Root Cause: Unrolling into Trees

The Computation Tree

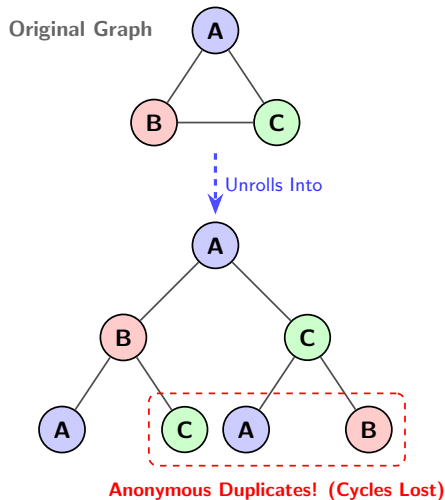
- GNNs aggregate messages recursively (k layers = k -hop tree).
- Nodes only see a multiset of features.

The Anonymity Problem

- Standard GNNs do not track exact edge identities or back-edges.
- Nodes act as anonymous in the tree.

Topological Blindness

- Cycles and paths are unrolled into identical tree structures.
- GNNs cannot distinguish shared neighbors from separate paths.



The Weisfeiler-Lehman (1-WL) Test

A Classical Benchmark

- 1-dimensional Weisfeiler-Lehman (1-WL) algorithm (1960s).
- A classical heuristic for testing graph isomorphism.

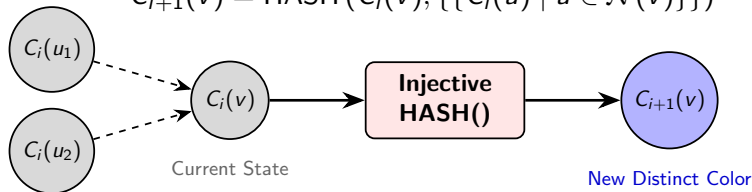
Color Refinement Algorithm

- Assigns a discrete label (“color”) to each node.
- Iteratively updates color based on neighbors’ colors.

The Hashing Mechanism

- Uses an **injective** hash: Unique neighbor multisets yield unique new colors.

$$C_{i+1}(v) = \text{HASH}(C_i(v), \{\{C_i(u) \mid u \in \mathcal{N}(v)\}\})$$



Walkthrough: 1-WL Color Refinement

Iteration 0: Initialization

- All nodes start with identical color.

Iteration 1: Aggregation

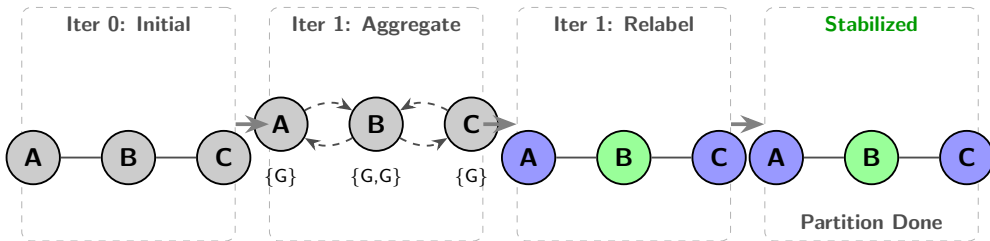
- Nodes gather neighbor colors.
- Deg-1 nodes: $\text{Hash}(G, \{G\})$
- Deg-2 nodes: $\text{Hash}(G, \{G, G\})$

Iteration 1: Relabeling

- New distinct colors assigned based on the unique multiset hash.

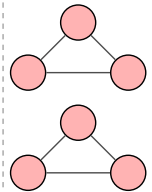
Termination

- Stop when histograms differ, or when color partitions stabilize.



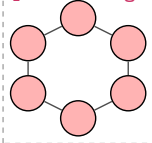
The 1-WL Failure Case

G_1 : Two Triangles



Identical 1-WL Hash!

G_2 : One Hexagon



The Incompleteness of 1-WL

- 1-WL is a powerful heuristic, but not perfect.
- It fails on highly symmetric graph structures.

Triangles vs. Hexagons

- G_1 : Two disconnected 3-cycles (Triangles).
- G_2 : One 6-cycle (Hexagon).
- *Result*: All nodes have degree 2. 1-WL colors them identically forever.

Engagement Question

- *If a financial network relies purely on the 1-WL test, could it distinguish three separate instances of simple 3-person fraud, versus one massive 9-person money laundering ring?*

The Grand Theorem: The Expressivity Ceiling

The Equivalence Theorem (Xu et al., Morris et al., 2019)

Any standard message-passing GNN is at most as powerful as the 1-WL test in distinguishing graph structures.

1-WL: $\text{Hash}(C(v), \{C(u)\})$

Discrete Colors
Injective Hash

Neural Realization

GNN: $\text{Update}(h(v), \text{Aggregate}(\{h(u)\}))$

Continuous Vectors
Neural Networks

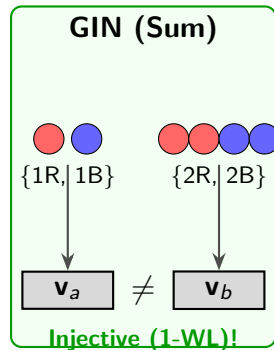
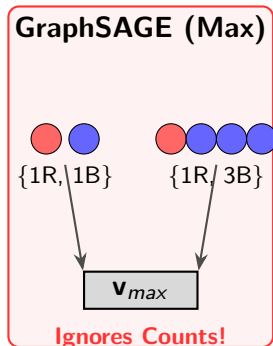
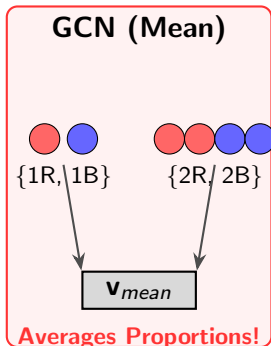
The Formal Limit

- If 1-WL evaluates two graphs as identical, a GNN will **provably** compute the exact same embedding for them.
- Standard GNNs are permanently blind to cycles and cliques.

Why GCN and GraphSAGE are Worse than 1-WL

The Injectivity Requirement

- To reach the 1-WL limit, the GNN Aggregate function must be *injective* over multisets.



From Algorithmic Heuristics to Formal Logic

The Limit of Algorithms

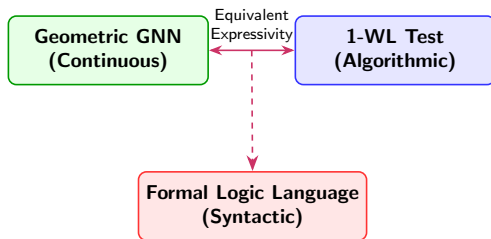
- 1-WL is a procedural heuristic. Shows *when* GNNs fail, not *what* they see.

The Return to Week 2

- Map continuous MPNN to relational logic to find its exact equivalent language.

Logical Classifiers for Nodes

- Evaluate GNNs as Boolean classifiers. Is there a formula replicating this decision?



First-Order Logic with Counting (FOC)

First-Order Logic (FOL) on Graphs

- **Variables:** x, y, z (representing nodes).
- **Predicates:** $E(x, y)$ (Edge exists), $Red(x)$ (Node attributes).

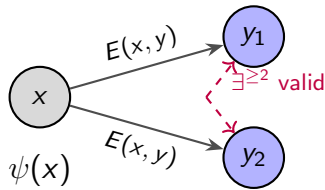
Adding Counting Quantifiers (FOC)

- Standard logic uses $\exists$ (exists) and $\forall$ (for all).
- GNN Aggregators (Sum/Mean) require counting: $\exists^{\geq N}$ (exists at least N).

An FOC Formula Example:

$$\psi(x) := \exists^{\geq 2} y (E(x, y) \wedge Blue(y))$$

- *Meaning:* Node x has at least 2 blue neighbors.



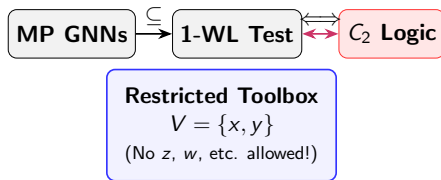
The C_2 Limit: The Logic of 1-WL

The Equivalence Theorem (Cai et al., 1992)

- Two nodes are colored identically by the 1-WL test *if and only if* they satisfy the exact same formulas in C_2 logic.

What is C_2 Logic?

- Also known as FOC_2 .
- It is First-Order Logic with counting, strictly restricted to a vocabulary of **exactly two variable symbols** (e.g., x and y).

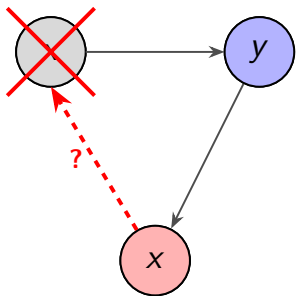


Engagement Question

- If our entire logical universe only allows us to use two variables (x and y), how can we mathematically describe a path that is three nodes long ($Node1 \rightarrow Node2 \rightarrow Node3$)?*

The Topological Blind Spot: Variable Recycling

Overwritten!



Goal: Describe a Triangle (3-Cycle)

- Verify loop: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

Step 1 & 2: The First Edge

- Anchor Node 1: x . Step to Node 2:
 $\exists y (E(x, y) \wedge \dots)$

Step 3: The Variable Recycling

- We must step to Node 3, but we are out of variables! Solution: We must overwrite x .
- $\dots \exists x (E(y, x) \wedge \dots)$

The Fatal Flaw (C_2 Blindness)

- Overwriting x deletes the origin's identity. Cannot verify the closing edge (Triangle $\equiv$ 3-hop tree).

The Exact GNN Boundary: AC-GNNs

Aggregate-Combine (AC) GNNs

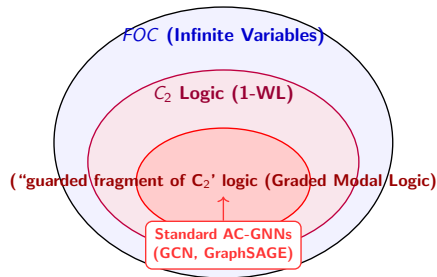
- Standard message-passing models (e.g., GCN).
- Node updates rely *strictly* on local neighborhood aggregation.

The Reality: AC-GNNs are weaker than C_2

- While bounded by C_2 , AC-GNNs cannot even express the entirety of C_2 logic!
- They fail on *global* properties (e.g., “Is there a red node *anywhere* in the graph?”).

The Precise Equivalence (Barceló et al., 2020)

- AC-GNNs $\iff$ “guarded fragment of C_2 ”



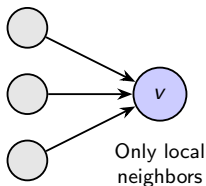
They can only verify properties within their local L -hop receptive field.

Reaching the C_2 Ceiling: The Readout Function

The Missing Piece

- How do we upgrade a GNN to capture the *entirety* of C_2 logic?
- We must grant nodes the ability to perceive the global graph state.

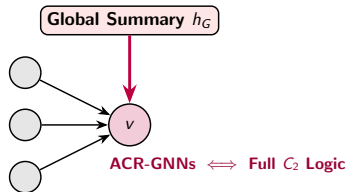
AC-GNN (Local Only)



Aggregate-Combine-Readout (ACR) GNNs

- Adds **Global Readout** to every layer.
- $h_G = \text{SumReadout}(\{h_v \mid \forall v \in V\})$
- Node Update incorporates: Local Aggregation + Node Memory + h_G .

ACR-GNN (Local + Global)



The Final Verdict on Standard Geometry

The Diagnosis is Complete

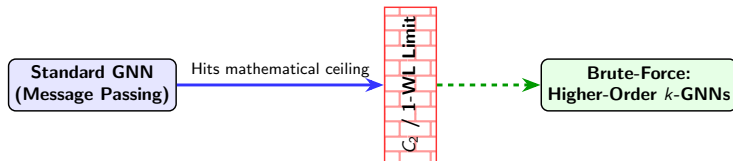
- Message passing is not a universal approximator.
- Standard GNNs $\rightarrow$ Local C_2 Logic (“guarded fragment”).
- GNNs with Readout $\rightarrow$ Global C_2 Logic.

The Unbreakable Barrier

- A 2-variable system (GNN) **cannot detect cycles or higher-order cliques**.
- *Consequence*: Pure geometry fails on complex relational motifs (e.g., fraud rings).

The Geometric Brute-Force Alternative

- To break this, researchers proposed scaling the geometry to match k -WL algorithms.



The Brute-Force Fix: Higher-Order k -GNNs

Generalizing the WL Test

- Higher-Order k -GNNs (Morris et al., 2019) simulate the k -dimensional WL algorithm.

The k -GNN Paradigm

- Instead of message passing between individual *nodes* (1-tuples)...
- ...we perform message passing between **subgraphs of k nodes** (k -tuples).

Expressivity Upgrade

- A 3-GNN operates on triplets of nodes.
- It successfully maps to C_3 logic (3 variables), allowing it to finally detect triangles!

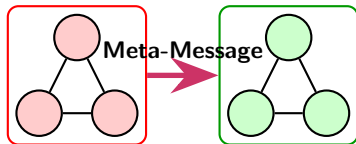
Standard 1-GNN

Node-to-Node



Higher-Order 3-GNN

Triplet-to-Triplet (C_3 Logic)



The Combinatorial Trap of Brute-Force Geometry

The Combinatorial Explosion

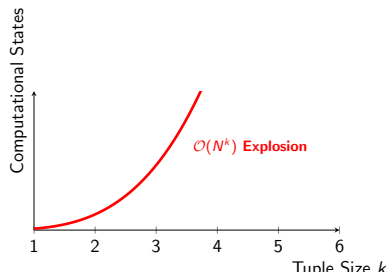
- A k -GNN requires instantiating state for *every possible k -tuple* in the graph.
- Memory and Compute Complexity: $\mathcal{O}(N^k)$.

Engagement Question

- *If we want to detect a 6-node money laundering ring using a brute-force k -GNN, we need a 6-GNN. How many 6-node tuples must be processed in a graph with just 1,000 users?*

The Answer:

- 1, 000, 000, 000, 000, 000, 000 tuples.
- **(1 Quintillion States!)**
- **Conclusion:** Pure geometric scaling is utterly unscalable for real enterprise databases.



The Paradigm Shift: From Bounds to Blueprints

The Role of Logic Thus Far

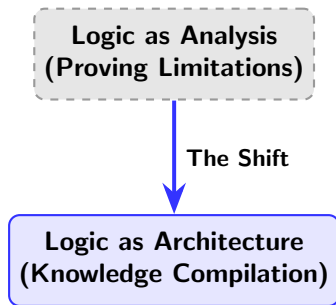
- In Module 3, we used logic strictly as an analytical tool (a measuring stick) to diagnose the mathematical boundaries of GNNs.

The Goal: Knowledge Compilation

- We must stop passively studying limits and actively **enforce** logic onto the network.
- We want to compile explicit human knowledge (rules) directly into the neural architecture.

Enter Neuro-Symbolic AI (NeSy)

- The field dedicated to marrying the explicit reasoning of logic with the differentiable perception of neural networks.



The Neuro-Symbolic Dream: A Historical Context

The Original Goal (1943)

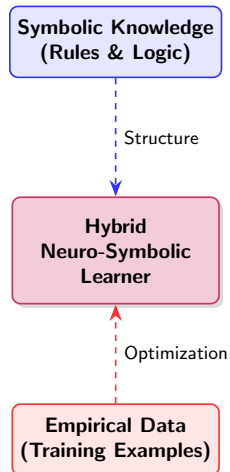
- McCulloch & Pitts: The first artificial neuron was explicitly designed to emulate boolean logic gates (AND/OR/NOT).

The Divergence

- *Symbolic AI*: Interpretable, rigid, brittle.
- *Deep Learning*: Robust, continuous, data-hungry black boxes.

The Hybrid Imperative

- How do we achieve the “middle ground”?
- **Goal:** Use explicit domain knowledge to give networks a “warm start”, reducing data dependency.



Visualizing Logic as a Perceptron

The Boolean Mapping

- How do we construct a logical AND gate using a standard perceptron?

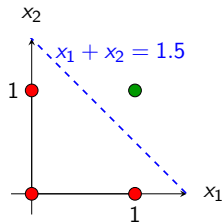
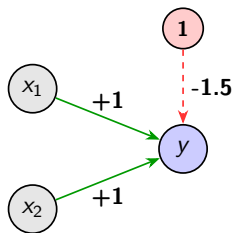
The Hard-Wired Weights

- Input literals: $x_1, x_2 \in \{0, 1\}$.
- Set positive weights: $w_1 = 1, w_2 = 1$.
- Set the Bias threshold to $\theta = -1.5$.

The Step Activation

$$y = \text{Step}(x_1 + x_2 - 1.5)$$

- If $x_1 = 1, x_2 = 0 \implies -0.5 \implies y = 0$.
- If $x_1 = 1, x_2 = 1 \implies +0.5 \implies y = 1$.



Knowledge-Based Artificial Neural Networks (KBANN)

The Structural Implementation

- Once we have continuous logic, how do we actually build the network?
- KBANN (Towell & Shavlik, 1994) physically wired logic rules into the MLP.

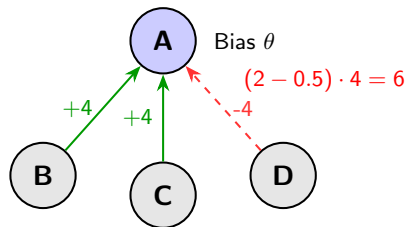
The Translation Mechanism

- **Nodes:** Logical literals map to neurons.
- **Weights:** $+v$ (e.g., 4) for positive literals, $-v$ (e.g., -4) for negated literals.

The Bias Threshold

- Bias θ is set to simulate a smoothed AND gate:
- $\theta = (P - 0.5) \cdot v$
- (where P is the number of positive antecedents)

$A :- B, C, \text{not } D$



The Differentiability Crisis

The Incompatibility of Pure Logic

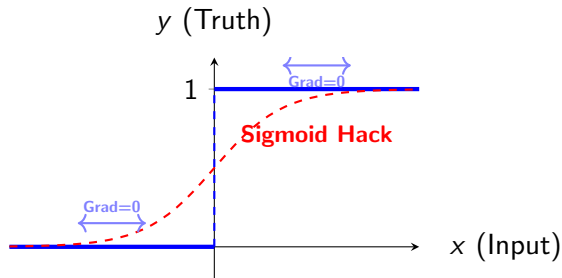
- Backpropagation requires continuous, non-zero derivatives.
- Discrete step functions jump instantly. The derivative is 0 everywhere.

Why not just use a Sigmoid?

- We *could* smooth the step function with a Sigmoid (like KBANN did).
- **The Issue:** It is an unprincipled mathematical hack.

The Failure of Sigmoid Logic

- Does not preserve logical axioms (e.g., associativity).



The Fix: Fuzzy Logic

Relaxing the Truth Domain

- We transition truth values from discrete $\{0, 1\}$ to the continuous real interval $[0, 1]$.
- A proposition possesses a *degree of truth* (e.g., 0.85 True).

Triangular Norms (T-Norms)

- A mathematical generalization of Boolean conjunction (AND).
- Operates smoothly on $[0, 1] \times [0, 1] \rightarrow [0, 1]$.
- Rigorously preserves logical axioms!



The Consequence

- By replacing strict logic gates with geometric T-norms, the entire logical framework becomes **fully backpropagatable**.

The Mathematics of Continuous Logic

Standard T-Norm Families

Logic	Operator	Product T-Norm	Gödel T-Norm	Łukasiewicz
AND	T-Norm ($\wedge$)	$a \cdot b$	$\min(a, b)$	$\max(0, a + b - 1)$
OR	T-Conorm ($\vee$)	$a + b - (a \cdot b)$	$\max(a, b)$	$\min(1, a + b)$
NOT	Negation ($\neg$)	$1 - a$	$1 - a$	$1 - a$

Choosing the Right Geometry

- **Product:** Smooth gradients, excellent for optimization, but scores vanish on long formulas (e.g., $0.9^{10} \approx 0.35$).
- **Gödel:** Preserves idempotence ($A \wedge A = A$), avoids vanishing scores, but min gradients can stall.

The Unifying Theory: T-Norm Generators

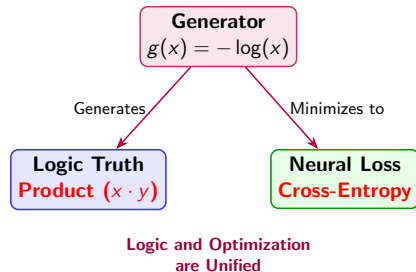
Constructing T-Norms

- T-norms are generated by a unary, strictly decreasing **Generator Function** $g(x)$.
- $T(x, y) = g^{-1}(\min(g(0^+), g(x) + g(y)))$

Logic $\iff$ Machine Learning Loss

- By setting the penalty for violating a logical constraint as $L = g(x)$, we recover standard ML loss functions!

Example: Cross-Entropy Link



Modern Synthesis: Logic as a Loss Function

Semantic Based Regularization (SBR)

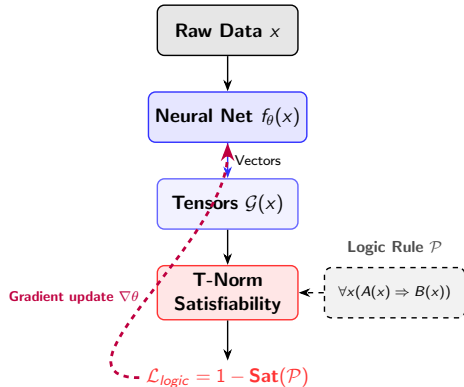
- Many frameworks no longer hard-wire rules into MLP connections.
- Logic is injected purely via the Loss Function.

Logic Tensor Networks (LTN)

- NN embeds data into grounded vectors $\mathcal{G}(x)$.
- T-Norms evaluate logic rules $\mathcal{P}$ over these vectors.

The Architectural Decoupling

- The network remains oblivious (MLP, CNN). Logic acts as a structural supervisor guiding the gradients.



The Propositional Bottleneck

The Shared Limitation

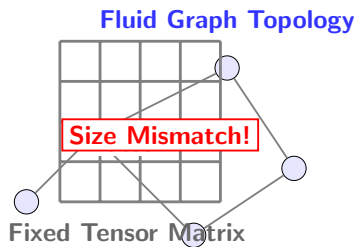
- But KBANN and standard NeSy operate on **propositional** or fixed-tensor inputs.

Topological Rigidity

- They assume a flat, fixed-size feature vector.
- They cannot natively handle dynamic, multi-relational graph topologies.

The Setup for Module 5

- To solve the topological blindness of GNNs, we need a framework that merges relational logic *directly* into a dynamic computation graph.



The Grand Dilemma: The Three Upgrades

1. The Representational Upgrade

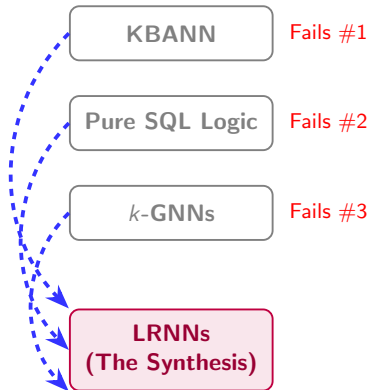
- Must escape flat matrices (Propositional).
- Must handle fluid, multi-hop graphs (**Relational**).

2. The Optimization Upgrade

- Must escape discrete step functions.
- Must use T-Norms for gradient descent (**Differentiable**).

3. The Expressive Upgrade

- Must escape the GNN 2-variable C_2 limit.
- Must avoid the $\mathcal{O}(N^k)$ explosion of k -GNNs (**Scalable Higher-Order**).



Lifted Relational Neural Networks (LRNNs)

The Neuro-Symbolic Synthesis

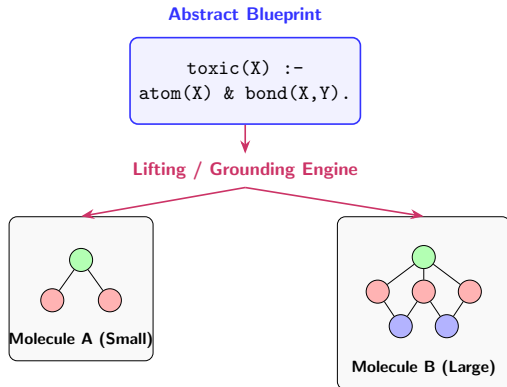
- Lifted Relational Neural Networks (LRNNs)
- Use Domain Logic to guide network construction instead of brute-forcing N^k .

What does “Lifted” mean?

- *Propositional*: Fixed matrices.
- *Lifted*: Abstract *relational* templates (variables $X, Y, Z, \dots$) as blueprints.

The Core Concept

- Use Weighted Horn Clauses to *dynamically* generate structure-tailored NNs.



The Grammar of LRNNs: Weighted Datalog

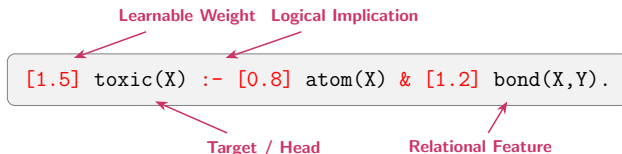
The Learning Template

- An LRNN is defined by a set of **Weighted Definite Clauses** (Datalog).
- Looks exactly like the rules mined in Week 2, but augmented with learnable parameters!

Anatomy of an LRNN Rule

$\{W_{out}\}$ target_concept(X) $:-$ $\{W_{in}\}$ feature(X, Y) & relation(Y, Z).

- $\{W\}$: Learnable tensor weights (scalars/matrices).
- $:-$: Logical implication (IF).
- $\&$: Logical conjunction (AND).



The Engine: Dynamic Grounding

The Grounding Process

- Grounding substitutes variables with concrete entities from a training example.

Herbrand Models (Computation Scope)

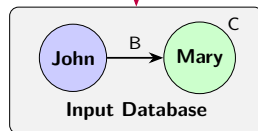
- Engine finds all valid logical substitutions in the DB.
- Constructs Least Herbrand Model: exact true facts derivable from the example.

Sparsity by Design

- Instantiates ONLY tuples that physically exist!
- Result:** $\mathcal{O}(N^k) \rightarrow \mathcal{O}(\text{Edges})$.

Template Rule

$A(X) :- B(X,Y), C(Y).$



Substitutions

$A(\text{John}) :- B(\text{John}, \text{Mary}), C(\text{Mary}).$

Grounded Rule (Herbrand Model)

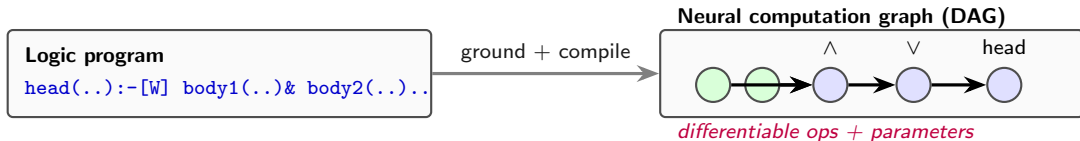
Translating Logic to Neural Nodes

Building the Computation Graph G_I

- Once grounded, the Herbrand model is deterministically translated into a differentiable Directed Acyclic Graph (DAG) - aka *neural network*.

The Structural Mapping:

Logical Construct	Neural Graph Node	Mathematical Operation
Ground Fact	Fact Node	Input feature vector/scalar.
Rule (AND)	Rule Node ($\wedge$)	T-Norm activation applied to weights and literals.
Multiple Proofs	Aggregation Node ($\vee$)	Permutation-invariant pooling (Max/Sum).



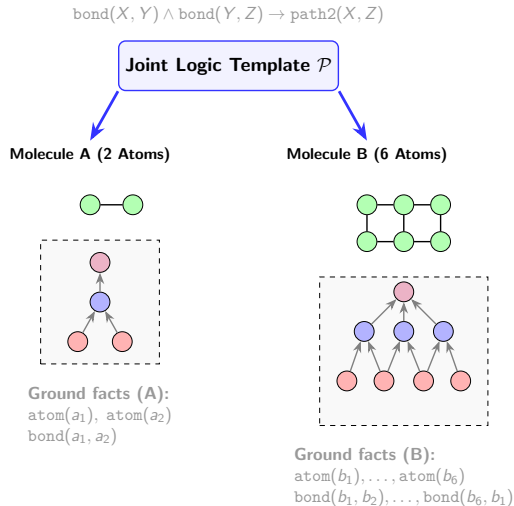
Visualizing the Unrolled LRNN

Dynamic Architecture

- Because each input graph has a different topology, the resulting Neural Computation Graph is uniquely shaped for *every single example*.

Example: Two Molecules

- Molecule A*: 2 atoms, 1 bond → Shallow, narrow neural net.
- Molecule B*: 6 atoms, ring structure → Deep, highly branched neural net.
- Crucial:** Both networks are generated from the **exact same logical template!**



The Secret to Generalization: Weight Tying

The Parameter Problem

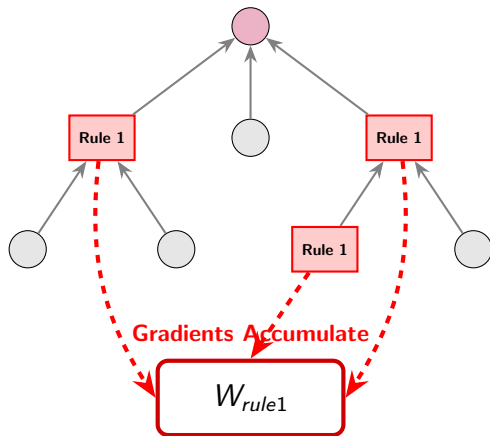
- How to learn if network shapes differ?

Lifted Weight Tying

- Parameters are attached to the **Logical** (the template), NOT to the specific structural nodes!

Global Sharing

- The weight W_{rule1} is shared across every single ground instantiation of that rule.
- Gradients accumulate from all topological locations to update the single abstract weight.



Simulating GNNs with LRNNs (Subsumption)

LRNNs are a Superset

- LRNNs can trivially simulate any standard message-passing GNN.

The GNN in Datalog

- A standard GNN layer equates to this simple LRNN rule:

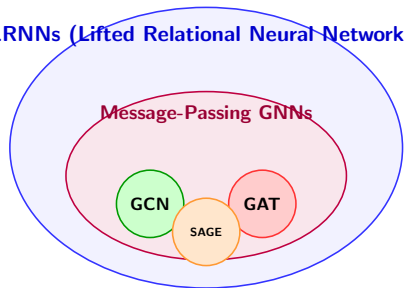
GNN message-passing rule

```
{W_layer1} h_1(X) :- node(Y) & edge(X,Y).
```

Customizing the Aggregation

- Settings like [activation=sigmoid, aggregation=avg] then replicate the exact GCN / GraphSAGE math.

LRNNs (Lifted Relational Neural Networks)



Breaking the C_2 Limit: Higher-Order Motifs

Escaping the 2-Variable Trap

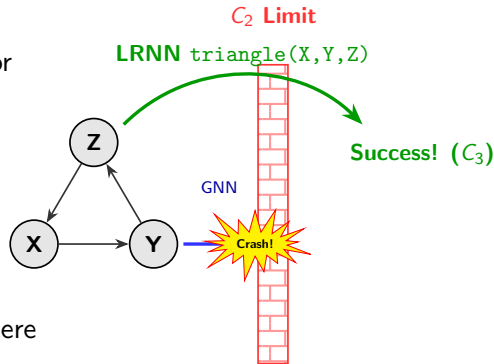
- GNNs are restricted to 2 variables (X, Y).
- LRNNs use arbitrary numbers of variables for complex motifs.

Defining a Motif (e.g., A Triangle)

$\{W\}$ triangle(X, Y, Z) :-
edge(X, Y), edge(Y, Z), edge(Z, X).

Targeted Higher-Order Passing

- The LRNN engine only grounds this rule where triangles *actually exist*.
- We achieve C_3 expressiveness without the $\mathcal{O}(N^3)$ brute-force explosion!



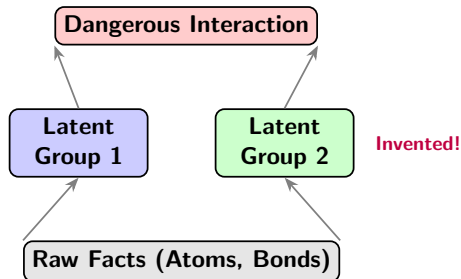
Hierarchical Latent Types (Inventing Predicates)

Beyond Feature Extraction: Predicate Invention

- LRNNs do not just pass messages; they can invent entirely new latent relational predicates/structures (Deep Relational Concepts) via gradient descent.
- **Analogy:** This is the logical equivalent of adding *Hidden Layers* in deep learning!

Hierarchical Logical Layers

```
{W_1} latent_group1(X) :-  
    toxic_atom(X) & high_energy(X).  
  
{W_2} latent_group2(Y) :-  
    safe_atom(Y).  
  
{W_3} danger_interact(X,Y) :-  
    latent_group1(X) &  
    latent_group2(Y) & edge(X,Y).
```



Week 5 Conclusion: The Grand Synthesis

1. The Formal Limits

- Continuous GNNs execute soft propositionalization.
- They are bounded by 1-WL / C_2 logic and are blind to complex topology.

2. The Differentiable Bridge

- We cannot optimize discrete logic gates.
- T-Norms relax boolean logic into a continuous space, enabling gradient descent.

3. The Neuro-Symbolic Solution

- Logic acts as the structural **Blueprint**.
- NNs act as differentiable **Instantiations**.
- *Architectures*: Logic Tensor Networks (LTNs), Lifted Relational NNs (LRNNs), and Probabilistic Logic (DeepProbLog).

